GPU-Accelerated Antimicrobial Resistance Pattern Analysis

Parallel Computing with Apple Metal Compute Shaders on Apple Silicon M3

Author: Sudharshini Manikandan | Target: Apple MacBook Air M3 (10-Core GPU)

PROJECT REPOSITORY & LIVE DEPLOYMENT LINKS:
- **GitHub Code Repository:** https://github.com/manisudhi/gpu
- **Live Deployed Web Application & Dashboard:** https://manisudhi.github.io/gpu/
- **Hardware Architecture:** Apple Silicon M3 (8-core CPU, 10-core Metal GPU, Unified Memory Architecture)

1. Executive Summary & Abstract

Antimicrobial resistance (AMR) creates a growing demand for high-throughput computational architectures capable of analyzing high-dimensional genomic resistance profiles, clustering emerging clinical clones, and predicting susceptibility without clinical diagnostic latency. This project implements an end-to-end, reproducible, GPU-native computing system engineered specifically for Apple Silicon (MacBook Air M3) using Apple Metal Compute Shaders (MSL) and PyTorch Metal Performance Shaders (MPS). The analytical pipeline processes 14,583 curated *Klebsiella pneumoniae* whole-genome sequencing isolates across 263 multi-hot numerical determinants. The Metal GPU kernels match 64-bit CPU reference calculations within $\varepsilon < 4.77 \times 10$■■, while delivering peak throughput of 5,446,186 records/second.

2. System Architecture & Branching Data Flow

The system uses a four-way branching GPU architecture where preprocessed contiguous float32 feature matrices feed into four concurrent GPU streams in Apple Silicon unified memory without PCIe transfer bottlenecks: (1) Supervised Phenotype Classification (PyTorch MPS Deep Neural Networks); (2) Parallel Query Similarity Search (Metal Compute Shaders + GPU Top-K); (3) Unsupervised K-Means Clustering (Metal Nearest-Centroid Assignment + GPU Reduction); and (4) Statistical Distance-to-Centroid Anomaly Detection (Metal Outlier Scoring).

3. Supervised Clinical Phenotype Prediction (PyTorch MPS)

Test set evaluation (N=694 confirmed antibiograms) across 10 critical antimicrobials:

Target Antibiotic	Tested	Accuracy	Precision	Recall	ROC-AUC
Ceftazidime	96	92.7%	1.000	91.4%	**0.9802**
Cefepime	86	90.7%	0.983	89.2%	**0.9802**
Ciprofloxacin	88	96.6%	0.985	97.1%	**0.9825**
Trimethoprim-SMX	83	90.4%	0.922	95.2%	**0.9482**
Gentamicin	88	86.4%	0.800	92.3%	**0.9314**
Ertapenem	98	81.6%	0.957	81.7%	0.8971
Imipenem	84	81.0%	0.862	88.9%	0.8964

Amikacin	88	78.4%	0.622	93.3%	0.8779
Meropenem	94	77.7%	0.877	81.4%	0.8720
Colistin	19	42.1%	0.375	85.7%	0.5238

4. Apple Silicon M3 Hardware Scaling Benchmarks

Synthetic expansion workloads (20,000 to 1,000,000 isolates) on MacBook Air M3 GPU:

Workload Size	GPU Kernel Time	Throughput	Memory	Characterization
20,000	4.02 ± 0.06 ms	4,977,481 rec/s	20.1 MB	Sub-5ms initial latency
50,000	9.00 ± 0.19 ms	**5,558,540 rec/s**	50.2 MB	Peak GPU saturation
100,000	18.12 ± 0.31 ms	**5,519,758 rec/s**	100.3 MB	Near-linear scaling
500,000	94.46 ± 2.39 ms	5,293,384 rec/s	501.6 MB	Under 100ms total latency
1,000,000	227.96 ± 33.18 ms	4,386,736 rec/s	1,003.3 MB	1GB Unified Memory Saturation

5. Unsupervised Clustering & Anomaly Detection

GPU K-Means (K=5) converged in 21 iterations. Cluster 1 isolated the canonical fluoroquinolone double-mutant clone (100% gyrA_S83I / parC_S80I, 6.2% MDR rate). Cluster 3 captured the major European extended-spectrum beta-lactamase (ESBL) clone (93.3% blaCTX-M-15). Distance-to-centroid anomaly detection flagged 146 candidate strains (1.00%) exceeding the 99th-percentile statistical cutoff (s > 4.608), led by isolate PDT000440856.1 (anomaly score: 5.6176) carrying 46 antimicrobial resistance determinants.

6. Reproduction & Execution Commands

```
# 1. Clone repository
git clone https://github.com/manisudhi/gpu.git
cd gpu/amr-gpu-ml
# 2. Setup virtual environment
python3 -m venv .venv && source .venv/bin/activate && pip install -r requirements.txt
# 3. Run full GPU test suite (10/10 tests pass)
make test
# 4. Execute end-to-end GPU pipeline (Phases 1 through 8)
make run
# 5. Launch interactive web dashboard
make app
```